

ROBOTICS - MINI PROJECT NUMBER 3

Arvin Mohammadi
student number: 810698303

December 9, 2022

Contents

0	INTRODUCTION	3
1	INSTALLING LINUX	3
2	INSTALLING ROS	3
3	CODING PYTHON SCRIPT	4

0 INTRODUCTION

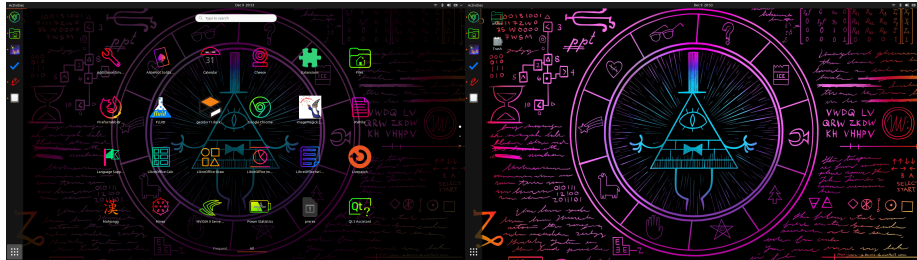
THIS is the third mini-project of the robotics course. The goal for this project is to use turtlesim from ROS software to write our own name (in my case it's ARVIN in all capital letters). The overview of the steps taken to do this projects are as explained below:

- Installing Linux OS on PC
- Installing Robotic Operating System(ROS) on Linux OS
- Coding the turtlesim python script.

1 INSTALLING LINUX

FIRST step of the project is installing Linux on PC, of course there is the option of using virtual box, but since I already had double-boot Linux and Windows, I decided to install ROS immediately. Though, the version of my Linux OS was Ubuntu 22.04 LTS (AKA Jellyfish release), ROS couldn't be installed. The latest release of Linux that is supported by ROS Noetic is Focal Fossa

Figure 1: Ubuntu Version 20.04 LTS - Focal Fossa



Linux OS with neon theme

2 INSTALLING ROS

THE second step of the project is installing ROS software. The Noetic release can be found on wiki.ros.org. As the tutorial demonstrated, the step of installation are pretty easy. We just have to copy and paste these commands:

```
$ sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc
```

```
$ sudo apt install curl
```

```
$ curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key ad
```

```
$ sudo apt update
```

```
$ sudo apt install ros-noetic-desktop-full
```

Figure 2: ROS Noetic



The logo of Noetic

These commands essentially lets the computer know that the download link is safe. and then download and install the package.

3 CODING PYTHON SCRIPT

THE third and final step is coding the python script. The overall logic for the python script is:

- Importing the libraries
- Functions including: Spawning turtle client, Moving forward, Rotating, Traveling through a curve
- Using the functions to spawn and move the turtles

As a sample of the functions, here is the written code for curved paths:

```
# function of traveling a curved path
def curve(radius, angle, direction):

# determining the message type
vel_msg = Twist()

# angular velocity of the turtle's movement
angular_vel = 0.5

# linear velocity of the turtle's movement
linear_vel = radius

# rate of sending the message
rate = rospy.Rate(100)

# setting the linear_velocity
vel_msg.linear.x = linear_vel
vel_msg.linear.y = 0
vel_msg.linear.z = 0

# setting the angular velocity
vel_msg.angular.x = 0
vel_msg.angular.y = 0
if direction == 0:
    vel_msg.angular.z = -angular_vel
else:
    vel_msg.angular.z = angular_vel

# the starting distance from initial position
current_angle = 0

# time zero
t0 = rospy.Time().now().to_sec()

rospy.loginfo("Turtle is rotating ... ")

#getting closer to the goal angle in a loop
while current_angle < angle:
```

```

# publish the message
pub.publish(vel_msg)

# updating the time
t1 = rospy.Time().now().to_sec() - t0

# calculating how much has the turtle rotated
current_angle = angular_vel*t1

# sleep for rate^-1
rate.sleep()

```

And the final result is as seen below:

Figure 3: Final Result



Drawing ARVIN with 5 turtles